

Towards Detection and Mitigation of Distributed Prompt Jailbreak Attacks on Large Language Models

Warshia Rubab, Roman Malik, Jawaria Maqsood
{F24609016, F24609031, F24609050 }@.nutech.edu.pk
Supervisor: Sir Babar Yousaf

Department of Computer Science, NUTECH, Islamabad, Pakistan

Abstract—Large Language Models (LLMs) face an escalating security threat from distributed prompt jailbreak attacks. In these scenarios, malicious actors bypass per-prompt safety filters by fracturing a harmful request across multiple independent sessions. The severity of this exploit was recently spotlighted by Waireus et al. through their Prompt, Divide, and Conquer (PDC) framework, which cleared state-of-the-art safety protocols with a 73.2% success rate. Subsequent evaluations by Li et al. via MalwareBench—a comprehensive suite of 3,520 adversarial prompts spanning 11 jailbreak methodologies—confirmed widespread vulnerabilities across 29 unique LLMs.

Our investigation highlights a critical gap in current defense literature: no existing security framework operates prior to the LLM processing its input. To close this vulnerability, we introduce PromptGuard, a three-stage, pre-inference middleware architecture. PromptGuard intercepts incoming traffic to identify fragmented syntactic patterns, evaluates multi-session context using Sentence-BERT (SBERT) embeddings coupled with cosine similarity, and isolates high-risk inputs via a fine-tuned DistilBERT classifier trained on MalwareBench data. To our knowledge, PromptGuard represents the initial middleware-driven defense engineered explicitly to counter distributed prompt jailbreaks within live LLM API environments.

Index Terms—Large Language Models, Jailbreak Detection, Prompt Injection, Distributed Attacks, AI Security, SBERT, MalwareBench, DistilBERT, Pre-inference Middleware.

I. INTRODUCTION

Large Language Models (LLMs) have swiftly transitioned from novel tech experiments to foundational infrastructure, anchoring decision-making systems in finance, healthcare, and national defense. Yet, as their societal footprint expands, they draw increasingly sophisticated adversarial exploitation. Foremost among these threats are jailbreak attacks—adversarial tactics engineered to trick an aligned model into generating dangerous, prohibited, or unethical outputs. While developers routinely update safety alignments, attack frameworks are evolving at a pace that traditional defense architectures struggle to match.

The distributed prompt jailbreak stands out as a highly elusive attack variation. Instead of delivering a blatantly malicious payload in a single message, an attacker breaks the core request down into small, benign-looking pieces. These fragments are then pushed simultaneously through independent LLM sessions. Because each individual prompt looks

entirely harmless on its own, it easily clears local safety filters. Once the model processes the inputs, the attacker simply harvests the separate responses and pieces them back together to reconstruct the forbidden output.

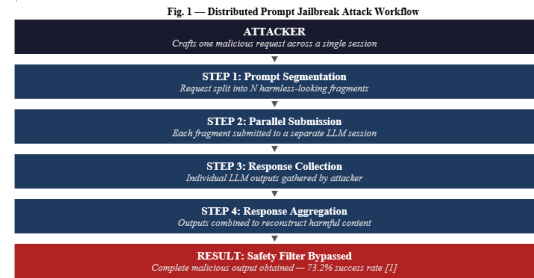


Fig. 1. Distributed prompt jailbreak attack workflow based on the PDC framework [1]. The attacker fragments a single malicious request into N individually benign prompts, submits each to a separate LLM session, and aggregates the responses to obtain the harmful output.

The underlying vulnerability stems from a basic structural reality: modern LLM guardrails are completely stateless. They judge each prompt inside a vacuum, devoid of cross-session context or historical memory. Waireus et al. exposed this blind spot by achieving a 73.2% bypass rate using their PDC framework across 10 distinct cybersecurity vulnerabilities. Further testing by Li et al. on MalwareBench underscored the scale of the crisis, proving that even top-tier commercial models like GPT-4o and Claude 3.5 fall victim to these coordinated strategies.

A close reading of recent literature reveals an uncomfortable truth: research remains heavily skewed toward offensive tactics and vulnerability benchmarking, while practical defense strategies are largely neglected. No existing framework sits ahead of the model to analyze prompt sequences before inference. This precise operational vacuum is what inspired PromptGuard. Designed as a pre-inference middleware layer, PromptGuard acts as an external security gatekeeper to catch distributed jailbreaks over live API endpoints, requiring zero modifications to the underlying LLM.

II. LITERATURE REVIEW

A. The PDC Framework (Waireus et al., 2025)

Waireus, Hussain, and Papadimitratos pioneered the structural logic behind distributed exploits through their Prompt, Divide, and Conquer (PDC) blueprint. The architecture relies on four main pillars: a prompt segmentation toolkit, a parallel multi-session dispatcher, a text aggregator, and an automated jury to grade response quality. While their evaluations demonstrated a stunning 73.2% success rate against aligned systems, the framework operates under three main constraints:

- a) *Infrastructure Assumptions::* It assumes attackers have unhindered access to multiple parallel sessions, a luxury often limited by modern rate-throttling protocols.
- b) *Scope Constraints::* The testing focused almost entirely on malicious code generation, ignoring adjacent risks like targeted disinformation, social engineering, and fraud scripts.
- c) *Defensive Deficit::* The paper serves exclusively as an attack demonstration; it diagnoses the systemic weakness but leaves the defensive space completely unaddressed.

B. MalwareBench (Li et al., 2025)

Li et al. expanded our understanding of LLM vulnerabilities by releasing MalwareBench, an extensive evaluation asset containing 3,520 adversarial prompts. This collection was grown from 320 foundational malicious mandates, stress-tested across 11 jailbreak profiles and evaluated against 29 unique language models. The benchmark offered definitive proof that even heavily guarded, closed-source models struggle against targeted multi-step prompt techniques. However, the contribution stops short of active mitigation. The creators treated MalwareBench strictly as an evaluation matrix rather than leveraging its rich data footprint to train a machine-learning-based defense classifier.

C. Collateral Research Frameworks

Parallel defense literature largely misses the mark regarding distributed threats. Zou et al. achieved notable success with universal, transferable adversarial suffixes, yet their methodology targets single-prompt string modifications rather than multi-session, fragmented text chains. Similarly, while Liu et al. compiled an extensive survey detailing prompt injection vectors across LLM-integrated ecosystems, their cataloged defenses rely heavily on internal weights alteration, intensive fine-tuning, or post-generation output analysis. These approaches remain functionally unviable for commercial entities interacting with proprietary models via third-party APIs.

D. Comparative Matrix

The comparative analysis above reveals that existing frameworks either lack defensive capabilities, remain strictly evaluative, or fail to address multi-session attack vectors. PromptGuard uniquely fills this gap with its three-stage pre-inference middleware architecture.

TABLE I
Comparative Overview of Prompt Security Frameworks: Deliverables and Limitations

Framework	Core Deliverable	Primary Vulnerability / Limitation
PDC Framework	Map out distributed exploits; 73.2% bypass rate across cyber vectors.	<i>Offers zero defensive logic; focuses strictly on code-based exploits.</i>
MalwareBench	3,520 adversarial prompt dataset across 11 jailbreaks and 29 LLMs.	<i>Strictly evaluative; data was never leveraged to build an active classifier.</i>
Zou et al.	Universal adversarial suffixes aimed at aligned systems.	<i>Fails to account for multi-session, split-prompt attack vectors.</i>
Liu et al.	Comprehensive overview of prompt injection defense trends.	<i>Defenses mandate model access; lacks middleware-level applicability.</i>
PromptGuard OURS	3-stage pre-inference middleware targeting distributed exploits.	Current implementation phase restricted to API-level integrations.

* Highlighted cells indicate critical gaps addressed by the proposed PromptGuard middleware architecture.

Fig. 2. Comparative Overview of Prompt Security Frameworks: Deliverables and Limitations. Highlighted cells indicate critical gaps addressed by the proposed PromptGuard middleware architecture.

III. IDENTIFIED CHALLENGES IN EXISTING DEFENSES

Our review of the current AI safety landscape highlights five unresolved industry pain points that PromptGuard is directly engineered to fix:

a) *The Blind Spot of Stateless Security Filters::* Traditional guardrails view incoming prompts as isolated events. If an attacker fractions a malicious script across a dozen distinct inputs, every single query looks clean on its own, easily evading detection due to a complete lack of cross-session memory.

b) *Absence of Pre-Inference Inspection Middleware::* Existing defensive tools trigger either during inference or right before the output is served (e.g., input sanitization or output scrapers). The research community lacks an inline proxy capable of parsing incoming token trends before the LLM compute cycle begins.

c) *Untapped Potential of MalwareBench::* Despite its rich, diverse collection of adversarial prompt mechanics, MalwareBench has not yet been utilized as a training foundation to construct a purpose-built defense classifier.

d) *Asymmetric Innovation and the Defense Deficit::* Defensive engineering is lagging dangerously behind offensive exploration. Frameworks like PDC have proven that distributed exploits work reliably in the wild, yet the community has yet to produce a corresponding enterprise-grade shield.

e) *Incompatibility with Proprietary APIs::* Organizations leveraging closed APIs (such as OpenAI, Google, or Anthropic) cannot apply defenses that require adjusting model weights or altering RLHF configurations. There is a critical, immediate need for a decoupled, plug-and-play defense layer.

IV. PROPOSED METHODOLOGY

PromptGuard is architected as a lightweight, pre-inference reverse-proxy middleware that stands directly between users and upstream LLM API gateways. It operates completely independently of the target model's internal architecture, making it universally compatible with both open-source backends and commercial APIs.

The system funnels queries through three distinct validation gates. A prompt must pass every checkpoint before it is cleared

for LLM processing. If any single stage detects a security breach with high confidence, the execution loop breaks, the query is blocked, and the event is written to a security log. This tiered workflow ensures that computationally demanding deep-learning models only evaluate inputs that have already tripped earlier structural alarms, keeping latency low.

A. Stage 1: Structural Anomaly Detector

The first gate scans each incoming prompt for four specific text-structure anomalies that statistically correlate with fragmented attack payloads:

- 1) **Syntactic Fragmentation:** Incomplete clauses that lack a standard subject-predicate relationship.
- 2) **Dangling Anaphors:** Isolated pronouns (e.g., "this," "the steps above," "it") that have no explicit linguistic referent within the current prompt scope.
- 3) **Abrupt Semantic Shifts:** Immediate, unprovoked transitions in topic matter.
- 4) **Contextual Underspecification:** Text strings that lack standalone utility and only make sense when paired with external information.

These signals are compiled into a weighted composite risk index. Queries that breach our configurable baseline are pushed up to Stage 2.

B. Stage 2: Cross-Session Context Analyzer

The second checkpoint uses a rolling memory buffer to retain incoming queries as dense vector representations generated by Sentence-BERT (SBERT). When Stage 1 flags a suspicious query, its vector embedding is evaluated against historical session records via cosine similarity. We use FAISS (Facebook AI Similarity Search) to ensure near-instantaneous nearest-neighbor lookups across the buffer cache. If the similarity trend indicates that the user is systematically building towards a unified, prohibited goal, the entire prompt cluster is escalated to Stage 3.

SBERT was selected over legacy bag-of-words or TF-IDF models because it interprets conceptual intent rather than exact keyword matches. This design choice prevents attackers from dodging detection by using simple lexical shifts or superficial rephrasing.

C. Stage 3: Intent Classification Engine

The final gate uses a specialized, fine-tuned DistilBERT classifier trained extensively on the MalwareBench dataset. The architecture is trained across all 29 malware categories present in the benchmark, utilizing an 80/10/10 data distribution split for training, tuning, and testing. Inputs that exceed the definitive risk threshold are immediately rejected. To minimize disruption to legitimate workflows, borderline cases are routed to a human-in-the-loop review queue rather than being flatly denied.

DistilBERT was carefully selected for this stage because of its balanced performance-to-speed profile. In live enterprise environments, any defense mechanism that introduces noticeable lag will simply be turned off by administrators.

Our engineering target is an end-to-end processing footprint under 100ms under standard API workloads.

D. Countering Informed Adversarial Tactics

A robust security framework must anticipate attackers who understand how the defense operates. PromptGuard accounts for two main evasion techniques:

- **Time-Delayed Injections:** Attackers might space out prompt fragments over long periods to slip past the session window. We counter this by using a time-decayed rolling memory buffer, ensuring that historical context retains a lingering suspicion footprint over time.
- **Semantic Noise Insertion:** Attackers might insert random conversational fluff between malicious fragments to distort cosine similarity metrics. However, doing so naturally introduces the exact syntactic anomalies and abrupt topic shifts that Stage 1 is specifically optimized to intercept.

E. Software Architecture and Tech Stack

PromptGuard will be constructed as a FastAPI-based Python reverse proxy. State persistence and similarity operations will be managed via a FAISS vector index, while model serving and tokenization tasks will run on the Hugging Face Transformers library. Every component is designed to be fully modular, allowing security teams to fine-tune threshold values dynamically without taking the system offline.

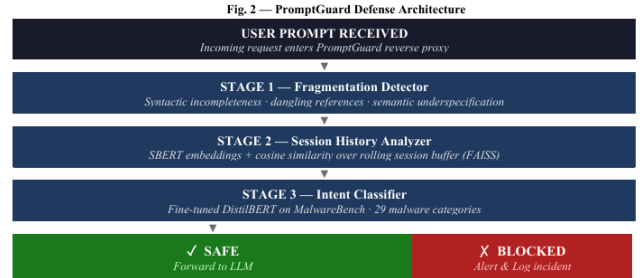


Fig. 3. PromptGuard three-stage defense architecture.

All prompts pass through Stage 1 (structural fragmentation detection), Stage 2 (SBERT semantic session analysis via FAISS), and Stage 3 (DistilBERT intent classification on MalwareBench) before reaching the LLM. Flagged inputs are blocked and logged.

V. EVALUATION PLAN

Our evaluation framework tests PromptGuard against both the full MalwareBench suite (covering 11 jailbreak vectors across 29 malware categories) and a custom set of simulated, multi-session PDC attacks designed to replicate real-world exploitation attempts. The success of the prototype will be measured against four explicit performance benchmarks:

- **Classification Accuracy:** Achieve an F_1 -score ≥ 0.85 across all MalwareBench threat classifications using the designated testing partition.

- **System Usability:** Maintain a False Positive Rate (FPR) $< 5\%$ when processing a standard corpus of benign user requests, ensuring uninterrupted operations for legitimate users.
- **Latency Profile:** Ensure end-to-end processing speeds remain under 100 ms at the 95th percentile under standard enterprise load conditions.
- **Defensive Efficacy:** Achieve a PDC Neutralization Rate $\geq 80\%$ against simulated distributed attacks, blocking the exploit before the attacker can assemble the complete malicious output.

Our baseline comparisons will measure PromptGuard against three distinct setups: direct, unprotected LLM access; standard, keyword-based regex filters; and isolated single-stage iterations of our own system. This ablation testing strategy will isolate the exact security value and computational overhead introduced by each individual stage.

VI. EXPECTED OUTCOMES

This research initiative expects to deliver the following tangible contributions to the AI safety landscape:

- A fully functional, production-ready PromptGuard reverse-proxy prototype capable of securing any upstream LLM API.
- A specialized DistilBERT intent classifier optimized explicitly for distributed prompt variants marking a novel defensive reuse of the MalwareBench dataset.
- A curated, labeled dataset mapping paired session prompts (both benign and adversarial) to support broader academic research into multi-session LLM security.
- Empirical data establishing definitive benchmarks for pre-inference, session-aware mitigation strategies.
- A comprehensive academic paper detailing our findings, prepared for submission to leading AI security conferences.
- A conceptual blueprint to extend PromptGuard’s defensive logic to multimodal pipelines (vision-language inputs) and voice-driven LLM interfaces.

VII. PROJECT TIMELINE

Ph.	Duration	Activities
1	Weeks 1–2	Deep literature review; finalise system requirements and adversarial threat model
2	Weeks 3–4	MalwareBench dataset preparation; train and validate baseline DistilBERT classifier
3	Weeks 5–6	Develop and unit-test Stage 1 (fragmentation detector) and Stage 2 (SBERT session analyser)
4	Weeks 7–8	Integrate Stage 3 classifier; build complete PromptGuard pipeline; integration testing
5	Weeks 9–10	Simulate PDC-style attacks; run full evaluation (F1, FPR, latency, PDC neutralisation rate)
6	Weeks 11–12	Write final research paper; prepare presentation slides; submit all deliverables

Fig. 4. Project timeline showing six phases over 12 weeks for PromptGuard development and evaluation.

The timeline is structured into six distinct phases: (1) literature review and requirements, (2) dataset preparation and baseline training, (3) core development of Stages 1 and 2, (4) pipeline integration of Stage 3, (5) adversarial simulation and benchmarking, and (6) final paper compilation and submission.

VIII. CONCLUSION

Our analysis of the LLM security space highlights a critical vulnerability: while attack frameworks like PDC and datasets like MalwareBench prove that distributed prompt jailbreaks are highly effective, existing defenses are entirely blind to them. Because current safety solutions operate either during or after inference inside the model, organizations utilizing third-party commercial APIs are left completely exposed to multi-session exploits.

PromptGuard offers a practical, model-agnostic solution to this problem. By combining structural anomaly tracking, SBERT-driven session history analysis, and targeted DistilBERT classification into an external, three-stage middleware layer, we provide a way to stop distributed exploits before they ever reach the model. This framework establishes an empirical foundation for a vital, emerging security paradigm: pre-inference, session-aware defense systems.

ACKNOWLEDGMENT

The authors express their sincere gratitude to Sir Babar Yousaf, Department of Computer Science, NUTECH, Islamabad, for his valuable guidance, constructive critiques, and steadfast encouragement throughout the formulation of this research framework.

REFERENCES

- [1] K. Greshake, S. Abdelnabi, S. Dutta, et al., “Prompt Injection Attacks and Defenses in LLM-Integrated Applications,” *arXiv preprint arXiv:2302.12173*, 2023.
- [2] H. Li, H. Gao, Z. Zhao, Z. Lin, J. Gao, and X. Li, “LLMs Caught in the Crossfire: Malware Requests and Jailbreak Challenges,” *arXiv preprint arXiv:2506.10022*, 2025.
- [3] A. Zou, Z. Wang, N. Carlini, et al., “Universal and Transferable Adversarial Attacks on Aligned Language Models,” *arXiv preprint arXiv:2307.15043*, 2023.
- [4] K. Greshake, S. Abdelnabi, S. Dutta, et al., “Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection,” *arXiv preprint arXiv:2302.12173*, 2023.